

Django 6.0 — Tüm Kod Örnekleri (Sırasıyla)

Sınıf Tabanlı Görünümler — Türkçe Çeviri

Bölüm 1: Sınıf Tabanlı Görünümler

Örnek 1: *URLconf* — *as_view()* ile *TemplateView*

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path("about/", TemplateView.as_view(template_name="about.html")),
]
```

Örnek 2: *some_app/views.py* — *AboutView*

```
# some_app/views.py
from django.views.generic import TemplateView

class AboutView(TemplateView):
    template_name = "about.html"
```

Örnek 3: *urls.py* — *AboutView* bağlantısı

```
# urls.py
from django.urls import path
from some_app.views import AboutView

urlpatterns = [
    path("about/", AboutView.as_view()),
]
```

Örnek 4: *urls.py* — *BookListView*

```
from django.urls import path
from books.views import BookListView

urlpatterns = [
    path("books/", BookListView.as_view()),
]
```

Örnek 5: *views.py* — *BookListView* (HEAD desteği)

```
from django.http import HttpResponse
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    model = Book

    def head(self, *args, **kwargs):
        last_book = self.get_queryset().latest("publication_date")
        response = HttpResponse(
            # RFC 1123 tarih biçimi.
            headers={
                "Last-Modified": last_book.publication_date.strftime(
                    "%a, %d %b %Y %H:%M:%S GMT"
                )
            },
        )
        return response
```

Örnek 6: *views.py* — *AsyncView* (asenكرون görünüm)

```
import asyncio
from django.http import HttpResponse
from django.views import View
```

```
class AsyncView(View):
    async def get(self, request, *args, **kwargs):
        # await ile G/Ç işlemleri gerçekleştirin.
        await asyncio.sleep(1)
        return HttpResponse("Merhaba, asenkron dünya!")
```

Bölüm 2: Sınıf Tabanlı Görünümlere Giriş

Örnek 7: *views.py* — İşlev tabanlı GET görünümü

```
from django.http import HttpResponse

def my_view(request):
    if request.method == "GET":
        # <view mantığı>
        return HttpResponse("result")
```

Örnek 8: *views.py* — Sınıf tabanlı GET görünümü (*MyView*)

```
from django.http import HttpResponse
from django.views import View

class MyView(View):
    def get(self, request):
        # <view mantığı>
        return HttpResponse("result")
```

Örnek 9: *urls.py* — *MyView* bağlantısı

```
# urls.py
from django.urls import path
from myapp.views import MyView

urlpatterns = [
    path("about/", MyView.as_view()),
]
```

Örnek 10: *views.py* — *GreetingView* (sınıf özniteliği)

```
from django.http import HttpResponse
from django.views import View

class GreetingView(View):
    greeting = "Good Day"

    def get(self, request):
        return HttpResponse(self.greeting)
```

Örnek 11: *views.py* — *MorningGreetingView* (alt sınıf)

```
class MorningGreetingView(GreetingView):
    greeting = "Morning to ya"
```

Örnek 12: *urls.py* — *as_view()* ile öznitelik yapılandırma

```
urlpatterns = [
    path("about/", GreetingView.as_view(greeting="G'day")),
]
```

Örnek 13: *views.py* — İşlev tabanlı form görünümü (*myview*)

```
from django.http import HttpResponseRedirect
from django.shortcuts import render

from .forms import MyForm
```

```
def myview(request):
    if request.method == "POST":
        form = MyForm(request.POST)
        if form.is_valid():
            # <form temizlenmiş verilerini işle>
            return HttpResponseRedirect("/success/")
        else:
            form = MyForm(initial={"key": "value"})

    return render(request, "form_template.html", {"form": form})
```

Örnek 14: *views.py* — *Sınıf tabanlı form görünümü (MyFormView)*

```
from django.http import HttpResponseRedirect
from django.shortcuts import render
from django.views import View

from .forms import MyForm

class MyFormView(View):
    form_class = MyForm
    initial = {"key": "value"}
    template_name = "form_template.html"

    def get(self, request, *args, **kwargs):
        form = self.form_class(initial=self.initial)
        return render(request, self.template_name, {"form": form})

    def post(self, request, *args, **kwargs):
        form = self.form_class(request.POST)
        if form.is_valid():
            # <form temizlenmiş verilerini işle>
            return HttpResponseRedirect("/success/")

        return render(request, self.template_name, {"form": form})
```

Örnek 15: *urls.py* — *URLconf'ta dekoratör kullanımı*

```
from django.contrib.auth.decorators import login_required, permission_required
from django.views.generic import TemplateView

from .views import VoteView

urlpatterns = [
    path("about/",
        login_required(TemplateView.as_view(template_name="secret.html"))),
    path("vote/", permission_required("polls.can_vote") (VoteView.as_view()))],
```

Örnek 16: *views.py* — *ProtectedView (@method_decorator dispatch)*

```
from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.views.generic import TemplateView

class ProtectedView(TemplateView):
    template_name = "secret.html"

    @method_decorator(login_required)
    def dispatch(self, *args, **kwargs):
        return super().dispatch(*args, **kwargs)
```

Örnek 17: *views.py* — *ProtectedView (sınıf düzeyinde dekoratör)*

```
@method_decorator(login_required, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"
```

Örnek 18: *views.py* — Çoklu dekoratör (liste/tuple yöntemi)

```
decorators = [never_cache, login_required]

@method_decorator(decorators, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"

@method_decorator(never_cache, name="dispatch")
@method_decorator(login_required, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"
```

Bölüm 3: Yerleşik Sınıf Tabanlı Genel Görünümler

Örnek 19: *models.py* — *Publisher*, *Author*, *Book* modelleri

```
# models.py
from django.db import models

class Publisher(models.Model):
    name = models.CharField(max_length=30)
    address = models.CharField(max_length=50)
    city = models.CharField(max_length=60)
    state_province = models.CharField(max_length=30)
    country = models.CharField(max_length=50)
    website = models.URLField()

    class Meta:
        ordering = ["-name"]

    def __str__(self):
        return self.name

class Author(models.Model):
    salutation = models.CharField(max_length=10)
    name = models.CharField(max_length=200)
    email = models.EmailField()
    headshot = models.ImageField(upload_to="author_headshots")

    def __str__(self):
        return self.name

class Book(models.Model):
    title = models.CharField(max_length=100)
    authors = models.ManyToManyField("Author")
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    publication_date = models.DateField()
```

Örnek 20: *views.py* — *PublisherListView*

```
# views.py
from django.views.generic import ListView
from books.models import Publisher

class PublisherListView(ListView):
    model = Publisher
```

Örnek 21: *urls.py* — *PublisherListView* bağlantısı

```
# urls.py
from django.urls import path
from books.views import PublisherListView
```

```
urlpatterns = [
    path("publishers/", PublisherListView.as_view()),
]
```

Örnek 22: *publisher_list.html — Şablon örneği*

```
{% extends "base.html" %}

{% block content %}
    <h2>Yayıncılar</h2>
    <ul>
        {% for publisher in object_list %}
            <li>{{ publisher.name }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Örnek 23: *views.py — context_object_name ile özelleştirme*

```
# views.py
from django.views.generic import ListView
from books.models import Publisher

class PublisherListView(ListView):
    model = Publisher
    context_object_name = "my_favorite_publishers"
```

Örnek 24: *views.py — PublisherDetailView (ekstra bağlam)*

```
from django.views.generic import DetailView
from books.models import Book, Publisher

class PublisherDetailView(DetailView):
    model = Publisher

    def get_context_data(self, **kwargs):
        # Önce temel uygulamayı çağır
        context = super().get_context_data(**kwargs)
        # Tüm kitapların QuerySet'ini ekle
        context["book_list"] = Book.objects.all()
        return context
```

Örnek 25: *views.py — PublisherDetailView (queryset ile)*

```
from django.views.generic import DetailView
from books.models import Publisher

class PublisherDetailView(DetailView):
    context_object_name = "publisher"
    queryset = Publisher.objects.all()
```

Örnek 26: *views.py — BookListView (yayın tarihine göre sıralama)*

```
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    queryset = Book.objects.order_by("-publication_date")
    context_object_name = "book_list"
```

Örnek 27: *views.py — AcmeBookListView (filtrelenmiş queryset)*

```
from django.views.generic import ListView
from books.models import Book

class AcmeBookListView(ListView):
    context_object_name = "book_list"
```

```
queryset = Book.objects.filter(publisher__name="ACME Publishing")
template_name = "books/acme_list.html"
```

Örnek 28: *urls.py — Dinamik filtreleme URLconf*

```
# urls.py
from django.urls import path
from books.views import PublisherBookListView

urlpatterns = [
    path("books/<publisher>/", PublisherBookListView.as_view()),
]
```

Örnek 29: *views.py — PublisherBookListView (get_queryset)*

```
# views.py
from django.shortcuts import get_object_or_404
from django.views.generic import ListView
from books.models import Book, Publisher

class PublisherBookListView(ListView):
    template_name = "books/books_by_publisher.html"

    def get_queryset(self):
        self.publisher = get_object_or_404(Publisher,
name=self.kwargs["publisher"])
        return Book.objects.filter(publisher=self.publisher)
```

Örnek 30: *views.py — get_context_data ile yayıncıyı bağlama ekleme*

```
# ...

def get_context_data(self, **kwargs):
    # Önce temel uygulamayı çağır
    context = super().get_context_data(**kwargs)
    # Yayıncıyı ekle
    context["publisher"] = self.publisher
    return context
```

Örnek 31: *models.py — Author (last_accessed alanıyla)*

```
# models.py
from django.db import models

class Author(models.Model):
    salutation = models.CharField(max_length=10)
    name = models.CharField(max_length=200)
    email = models.EmailField()
    headshot = models.ImageField(upload_to="author_headshots")
    last_accessed = models.DateTimeField()
```

Örnek 32: *urls.py — AuthorDetailView bağlantısı*

```
from django.urls import path
from books.views import AuthorDetailView

urlpatterns = [
    # ...
    path("authors/<int:pk>/", AuthorDetailView.as_view(), name="author-detail"),
]
```

Örnek 33: *views.py — AuthorDetailView (get_object geçersiz kılma)*

```
from django.utils import timezone
from django.views.generic import DetailView
from books.models import Author

class AuthorDetailView(DetailView):
    queryset = Author.objects.all()
```

```
def get_object(self):
    obj = super().get_object()
    # Son erişim tarihini kaydet
    obj.last_accessed = timezone.now()
    obj.save()
    return obj
```

Bölüm 4: Sınıf Tabanlı Görünümlerle Form İşleme

Örnek 34: *forms.py* — *ContactForm*

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField()
    message = forms.CharField(widget=forms.Textarea)

    def send_email(self):
        # self.cleaned_data sözlüğünü kullanarak e-posta gönder
        pass
```

Örnek 35: *views.py* — *ContactFormView* (*FormView* kullanımı)

```
from myapp.forms import ContactForm
from django.views.generic.edit import FormView

class ContactFormView(FormView):
    template_name = "contact.html"
    form_class = ContactForm
    success_url = "/thanks/"

    def form_valid(self, form):
        # Bu yöntem geçerli form verisi POST edildiğinde çağrılır.
        # HttpResponse döndürmelidir.
        form.send_email()
        return super().form_valid(form)
```

Örnek 36: *models.py* — *Author* (*get_absolute_url* ile)

```
from django.db import models
from django.urls import reverse

class Author(models.Model):
    name = models.CharField(max_length=200)

    def get_absolute_url(self):
        return reverse("author-detail", kwargs={"pk": self.pk})
```

Örnek 37: *views.py* — *AuthorCreateView*, *AuthorUpdateView*, *AuthorDeleteView*

```
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, DeleteView, UpdateView
from myapp.models import Author

class AuthorCreateView(CreateView):
    model = Author
    fields = ["name"]

class AuthorUpdateView(UpdateView):
    model = Author
    fields = ["name"]

class AuthorDeleteView(DeleteView):
```

```
model = Author
success_url = reverse_lazy("author-list")
```

Örnek 38: *urls.py — CreateView, UpdateView, DeleteView bağlantıları*

```
from django.urls import path
from myapp.views import AuthorCreateView, AuthorDeleteView, AuthorUpdateView

urlpatterns = [
    # ...
    path("author/add/", AuthorCreateView.as_view(), name="author-add"),
    path("author/<int:pk>/", AuthorUpdateView.as_view(), name="author-update"),
    path("author/<int:pk>/delete/", AuthorDeleteView.as_view(), name="author-
delete"),
]
```

Örnek 39: *models.py — Author (created_by yabancı anahtar ile)*

```
from django.contrib.auth.models import User
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length=200)
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)

    # ...
```

Örnek 40: *views.py — AuthorCreateView (form_valid ile kullanıcı ekleme)*

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic.edit import CreateView
from myapp.models import Author

class AuthorCreateView(LoginRequiredMixin, CreateView):
    model = Author
    fields = ["name"]

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

Örnek 41: *views.py — JsonableResponseMixin (içerik uyumu — temel)*

```
from django.http import JsonResponse
from django.views.generic.edit import CreateView
from myapp.models import Author

class JsonableResponseMixin:
    """
    Bir forma JSON desteği eklemek için Mixin.
    Nesne tabanlı FormView ile kullanılmalıdır (örn. CreateView)
    """

    def form_invalid(self, form):
        response = super().form_invalid(form)
        if self.request.accepts("text/html"):
            return response
        else:
            return JsonResponse(form.errors, status=400)

    def form_valid(self, form):
        response = super().form_valid(form)
        if self.request.accepts("text/html"):
            return response
        else:
            data = {
                "pk": self.object.pk,
            }
            return JsonResponse(data)
```



```
class AuthorCreateView(JsonableResponseMixin, CreateView):
    model = Author
    fields = ["name"]
```

Örnek 42: *views.py* — *JsonableResponseMixin* (*get_preferred_type* ile gelişmiş)

```
class JsonableResponseMixin:
    """
    Bir forma JSON desteği eklemek için Mixin.
    Nesne tabanlı FormView ile kullanılmalıdır (örn. CreateView).
    """

    accepted_media_types = ["text/html", "application/json"]

    def dispatch(self, request, *args, **kwargs):
        if request.get_preferred_type(self.accepted_media_types) is None:
            # Ortak format yok.
            return HttpResponse(
                status_code=406, headers={"Accept":
            ", ".join(self.accepted_media_types)}
        )

        return super().dispatch(request, *args, **kwargs)

    def form_invalid(self, form):
        response = super().form_invalid(form)
        accepted_type = self.request.get_preferred_type(self.accepted_media_types)
        if accepted_type == "text/html":
            return response
        elif accepted_type == "application/json":
            return JsonResponse(form.errors, status=400)

    def form_valid(self, form):
        response = super().form_valid(form)
        accepted_type = self.request.get_preferred_type(self.accepted_media_types)
        if accepted_type == "text/html":
            return response
        elif accepted_type == "application/json":
            data = {
                "pk": self.object.pk,
            }
            return JsonResponse(data)
```

Toplam: 42 kod örneği